

Research Paper Presentation
On
Performance Analysis of JVM Garbage
Collectors

Priyanshi Jain
Rakshita
Taniya
Divyanjali Singla
Rajat Takkar

jpriya1530@gmail.com
rakshitabhardwaj23@gmail.com
goyaltaniya40@gmail.com
divyanjalisingla@gmail.com
rajat.takkar@chitkara.edu.in

Supervised By : Dr. Lalit Kumar

Department of Computer Science and Engineering,
Chitkara University, Punjab

Memory management is a key element of modern software systems, especially those running on the Java Virtual Machine (JVM). Garbage Collection (GC) automatically reclaims unused memory, mitigating memory leaks and reducing the need for manual memory management. The JVM offers multiple GC algorithms, each with different trade-offs in latency, throughput, and memory efficiency. This research presents a comparative study of Serial GC, Parallel GC, G1 GC, ZGC, and Shenandoah under memory-intensive workloads. Key findings show that modern concurrent collectors (ZGC and Shenandoah) drastically reduce pause times, making them ideal for real-time and large-scale applications. G1 GC strikes a balance between throughput and latency, while Serial and Parallel GC exhibit limitations under heavy loads.

Key Prior Research

Dashti & Fedorova (2020): GPU-based GC offers no significant benefit due to memory bandwidth limitations

Kolokasis et al. (2021): TeraHeap reduces GC overhead in big data systems but requires JVM adjustments

Zhao et al. (2022): Parallel GC improves throughput and reduces pause time but adds system complexity

Islam et al. (2023): ZGC shows low pause times vs. Serial GC high latency using big data benchmarks

Chen et al. (2024): GC optimizations for ML applications improve responsiveness but add compute cost

Research Gaps

Synthetic benchmarks: Most studies use artificial workloads, limiting real-world applicability

No universal selector: No automated method exists to choose the best GC per application type

ML-based methods: Introduce overhead and remain unreliable for production systems

Four-Layer Modular Architecture

Workload Generation Layer: Memory-intensive workloads using DaCapo benchmarking suite and custom stress applications simulating real-world scenarios

JVM Execution Layer: Runs workloads with Serial GC, Parallel GC, G1 GC, ZGC, and Shenandoah configured with standardized heap sizes and JVM settings

Monitoring Module: Collects runtime statistics including heap usage, GC pause time CPU consumption, and throughput using JVisualVM and JConsole

Analysis & Evaluation Engine: Processes collected data to compute performance metrics identify trade-offs, and generate comparative reports for GC selection

System Workflow

Workflow: Workload Input → JVM Execution → GC → Monitoring → Data Collection Analysis → Results →

Experimental Setup

Hardware: Multi-core system running standard JVM; heap fixed at -Xms2g -Xmx2g

Collectors Tested: Serial GC, Parallel GC, G1 GC, ZGC, Shenandoah — identical system settings

Workload: 5 warm-up rounds + 15 measured rounds; 120 outer loops, 80 inner allocations, 256 KB block size

Performance Metrics

Throughput (%): Percentage of time application runs without GC pauses

Avg Pause Time (ms): Mean stop-the-world duration per GC collection cycle

CPU Usage (%): Processor overhead consumed by GC operations per round

Memory Before/After (MB): Heap usage before and after each GC cycle

GC Count and GC Time: Number of collection events and cumulative collection time per round

Development Environment

Language/Platform: Java on JVM — supports Serial, Parallel, G1, ZGC, and Shenandoah collectors

Benchmark Program: Custom GCResearchBenchmark.java — measures exec time, pause, throughput, CPU, GC stats

Key Components

captureGCStats(): Uses ManagementFactory.getGarbageCollectorMXBeans() to track GC count and time delta per round

getProcessCpuTimeNs(): Reads process CPU time via com.sun.management.OperatingSystemMXBean

printSummary(): Computes avg/min/max exec time, avg pause, throughput, CPU, GC stats across all rounds

Run Commands (Examples)

G1 GC: java -Xms2g -Xmx2g -XX:+UseG1GC GCResearchBenchmark

Serial GC: java -Xms2g -Xmx2g -XX:+UseSerialGC GCResearchBenchmark

Parallel GC: java -Xms2g -Xmx2g -XX:+UseParallelGC GCResearchBenchmark

Code Structure Overview

,printSummary() (lines 224-254): Accumulates exec time, pause, throughput, CPU memory, GC stats; tracks min/max exec times

captureGCStats() (lines 138-155): Queries all GarbageCollectorMXBean instances to get total GC collection count and time in ms

;getProcessCpuTimeNs() (lines 158-166): Casts OperatingSystemMXBean to sun flavor retrieves per-process CPU time in nanoseconds

Output Fields Per Round

Core metrics: Execution Time, Avg Pause Time, Throughput (%) — primary latency and throughput data

Resource metrics: CPU Usage %, GC Count Delta, GC Time Delta — overhead measurement

Memory metrics: Memory Before and After (MB), Checksum — heap behavior and correctness

G1 GC Final Summary (Measured Benchmark)

Avg Exec Time: 499.93ms | Min: 227ms | Max: 947ms across 15 rounds

Avg Pause Time: 5.08ms | Throughput: 95.07% | CPU Usage: 4.78%

| GC Stats: Avg GC Time: 22.27ms | Avg GC Count: 4.40 | Avg Mem Before: 378.85MB
After: 365.74MB

Comparative Summary (from Research)

,Serial GC: Avg Pause ~120ms | Throughput ~70% | CPU ~40% — single-threaded simple workloads

Parallel GC: Avg Pause ~95ms | Throughput ~82% | CPU ~75% — multi-threaded batch processing

G1 GC: Avg Pause ~45ms | Throughput ~88% | CPU ~68% — best general-purpose balance

ZGC: Avg Pause ~12ms | Throughput ~91% | CPU ~82% — ultra-low latency concurrent

Shenandoah: Avg Pause ~18ms | Throughput ~89% | CPU ~80% — low latency concurrent

G1 GC — Benchmark Output

G1 GC run with -XX:+UseG1GC | Heap: -Xms2g -Xmx2g | 5 warm-up + 15 measured rounds

```
GC Research Benchmark
Warm-up Rounds   : 5
Measured Rounds  : 15
Outer Loops      : 120
Inner Allocations: 80
Block Size       : 256 KB
=====
Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed
-----
Round           : 1
Execution Time  : 206 ms
Avg Pause Time  : 4.33 ms
Throughput      : 93.69 %
CPU Usage       : 7.08 %
GC Count Delta  : 3
GC Time Delta   : 13 ms
Memory Before   : 1150.16 MB
Memory After    : 826.52 MB
Checksum        : 275424
-----
Round           : 2
Execution Time  : 191 ms
```

Parallel GC — Benchmark Output

Parallel GC run with -XX:+UseParallelGC | Heap: -Xms2g -Xmx2g | 5 warm-up + 15 measured rounds

```
Memory Before   : 540.07 MB
Memory After    : 185.10 MB
Checksum        : 275424
-----
Round           : 15
Execution Time  : 306 ms
Avg Pause Time  : 4.75 ms
Throughput      : 93.79 %
CPU Usage       : 6.69 %
GC Count Delta  : 4
GC Time Delta   : 19 ms
Memory Before   : 185.10 MB
Memory After    : 396.22 MB
Checksum        : 275424
=====
FINAL SUMMARY
Average Exec Time : 499.93 ms
Minimum Exec Time : 227 ms
Maximum Exec Time : 947 ms
```

Serial GC — Benchmark Output

```
GC Research Benchmark
Warm-up Rounds   : 5
Measured Rounds  : 15
Outer Loops      : 120
Inner Allocations: 80
Block Size       : 256 KB
```

```
=====
Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed
```

```
-----
Round           : 1
Execution Time  : 205 ms
Avg Pause Time  : 6.25 ms
Throughput      : 87.80 %
CPU Usage       : 6.18 %
GC Count Delta  : 4
GC Time Delta   : 25 ms
Memory Before   : 147.86 MB
Memory After    : 382.23 MB
Checksum        : 275424
```

```
-----
Round           : 2
Execution Time  : 218 ms
Avg Pause Time  : 5.40 ms
```

printSummary() Method

printSummary() iterates Result list: accumulates exec, pause, throughput, CPU, memory, GC stats;
.tracks min/max exec time

```
224 private static void printSummary(List<Result> results) {  
225     double totalExec = 0;  
226     double totalPause = 0;  
227     double totalThroughput = 0;  
228     double totalCpu = 0;  
229     double totalMemBefore = 0;  
230     double totalMemAfter = 0;  
231     double totalGcTime = 0;  
232     double totalGcCount = 0;  
233  
234     long minExec = Long.MAX_VALUE;  
235     long maxExec = Long.MIN_VALUE;  
236  
237     for (Result r : results) {  
238         totalExec += r.execTimeMs;  
239         totalPause += r.avgPauseMs;  
240         totalThroughput += r.throughput;  
241         totalCpu += r.cpuUsage;  
242         totalMemBefore += bytesToMB(r.memBefore);  
243         totalMemAfter += bytesToMB(r.memAfter);  
244         totalGcTime += r.gcTimeMs;  
245         totalGcCount += r.gcCount;  
246  
247         if (r.execTimeMs < minExec) {  
248             minExec = r.execTimeMs;  
249         }  
250  
251         if (r.execTimeMs > maxExec) {  
252             maxExec = r.execTimeMs;  
253         }  
254     }  
255 }
```

captureGCStats() Method

captureGCStats() queries GarbageCollectorMXBeans to compute total GC collection count and total .collection time in ms

```
139     long totalCount = 0;
140     long totalTimeMs = 0;
141
142     for (GarbageCollectorMXBean bean : ManagementFactory.getGarbageCollectorMXBeans()) {
143         long c = bean.getCollectionCount();
144         long t = bean.getCollectionTime();
145
146         if (c != -1) {
147             totalCount += c;
148         }
149
150         if (t != -1) {
151             totalTimeMs += t;
152         }
153     }
154
155     return new GCStats(totalCount, totalTimeMs);
156 }
157
158 private static long getProcessCpuTimeNs() {
159     try {
160         com.sun.management.OperatingSystemMXBean osBean =
161             (com.sun.management.OperatingSystemMXBean) ManagementFactory.getOperatingSystemMXBean();
162         return osBean.getProcessCpuTime();
163     } catch (Exception e) {
164         return 0;
165     }
166 }
```

G1 GC — Final Summary Output

```
GC Research Benchmark  
Warm-up Rounds : 5  
Measured Rounds : 15  
Outer Loops : 120  
Inner Allocations: 80  
Block Size : 256 KB
```

```
=====
```

Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed

```
-----
```

Round : 1
Execution Time : 256 ms
Avg Pause Time : 3.80 ms
Throughput : 92.58 %
CPU Usage : 7.23 %
GC Count Delta : 5
GC Time Delta : 19 ms
Memory Before : 571.38 MB
Memory After : 276.39 MB
Checksum : 275424

```
-----
```

Round : 2
Execution Time : 227 ms
Avg Pause Time : 3.75 ms
Throughput : 93.39 %

Thank You